A REPORT
CLASS TIMETABLE GENERATOR
USING GREEDY AND DYNAMIC PROGRAMMING
ON BY

Name of the Students	**Registration No.**
CHIMATA NITHESSH	AP24110012053
POLISETTI JASHITH	AP24110012064
DUBA PHANI SASI SRIDHAR	AP24110011960
VARRE SAI SOHITH	AP24110012128
KATLAGUNTA MANOHAR	AP24110012036

Prepared in the partial fulfillment of the

Project Based Learning of Course CSE 206 – CODING SKILLS II

Under The Guidance Of

DR. PRAKASH

SRM UNIVERSITY, AP

APRIL- 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have contributed to the successful completion of this project.

First and foremost, we are deeply grateful to our Honourable Vice Chancellor **Prof. Ch. Satish Kumar sir** and **Dean of the School of Engineering and Sciences Prof. C V Tomy sir** and the Head of the Department for Computer Science and Engineering **Dr.Murali Krishna Enduri** PhD Associate Professor providing us with the opportunity and necessary facilities to undertake this project. Their constant encouragement and commitment to academic excellence have been a great source of motivation throughout this work.

I would like to extend my heartfelt thanks to my faculty Mentor **Dr. Prakash** PhD for their valuable guidance, insightful suggestions, and continuous support during the course of the project. Their expertise and professional insights have played a crucial role in shaping the outcome of this work.

I also wish to convey my sincere appreciation to all my Faculty members of the department for their encouragement, constructive feedback, and invaluable academic guidance. Their continuous supervision and advice have been instrumental in the successful completion of this project.

Finally, I would like to thank all those who have directly or indirectly helped me during this project — my colleagues, friends, and family — for their constant support, understanding, and cooperation.

With Regards

CH.Nitheesh (AP24110012053)

P.Jashith (AP24110012064)

V.Sohith (AP24110012128)

D.Sridhar (AP24110011960)

K.Manohar(AP24110012036)

CERTIFICATE

This is to certify that the project entitled has been successfully completed by POLISETTI JASHITH(Roll Number: AP24110012064), DUBA PHANI SASI SRIDHAR (Roll Number: AP24110011960),CHIMATA NITHEESH(Roll Number:AP24110012053) , VARRE SAI SOHITH (Roll Number:AP24110012128),KATLAGUNTA MANOHAR(ROLL NUMBER:AP24110012036) as a required academic component of the course **CSE 206: CODING SKILLS II** during Semester 4 of Academic Year 2025-26 in the Department of Computer Science and Engineering at SRM University, AP. This work was executed under the guidance of the undersigned and is deemed to have successfully met all course requirements as of 30/04/2026.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING, SRM UNIVERSITY AP,

NEERUKONDA, MANGALAGIRI-522240,

ANDHRAPRADESH, INDIA.

(Signature of the Course Instructor)

DR.PRAKASH,

PROFESSOR,

Department of Computer Science and Engineering,

SRM University,AP.

ABSTRACT

A class timetable plays a crucial role in organizing academic activities efficiently. However, preparing a timetable manually is time-consuming and often leads to conflicts such as uneven subject distribution, repetition of classes, or improper allocation of faculty and rooms. To overcome these issues, this project presents a **Smart Class Timetable Generator** using a combination of **Greedy Algorithm and constraint-based techniques inspired by Dynamic Programming**.

The system collects inputs such as subject names, faculty details, classroom information, and required weekly periods. Based on these inputs, it automatically generates a timetable for five working days with a fixed number of periods per day. The greedy approach assigns subjects to each time slot by making locally optimal decisions while satisfying constraints like limiting the number of classes per subject per day and avoiding consecutive repetition.

To ensure balanced scheduling and reduce conflicts, the system tracks subject allocations and applies constraints efficiently. A randomization technique is also used to introduce fairness and variation in timetable generation.

The generated timetable is clear and well-structured, displaying subject, faculty, and room details for each period. This approach reduces manual effort, improves accuracy, and ensures effective utilization of resources.

In conclusion, the project demonstrates how algorithmic techniques can be applied to solve real-world scheduling problems efficiently in educational institutions.

INTRODUCTION

In educational institutions, preparing a class timetable is an essential task that ensures smooth functioning of academic activities. A well-structured timetable helps in organizing subjects, assigning faculty, and managing classroom usage effectively. However, creating a timetable manually is often a challenging and time-consuming process. It requires careful planning to avoid issues such as overlapping classes, uneven distribution of subjects, and improper allocation of resources.

With the increasing number of courses, subjects, and faculty members, manual scheduling becomes inefficient and prone to errors. It is difficult to satisfy multiple constraints simultaneously, such as assigning the required number of periods for each subject, avoiding consecutive repetition of the same subject, and maintaining a balanced workload across the week. These challenges highlight the need for an automated and intelligent solution.

This project introduces a **Smart Class Timetable Generator** that uses algorithmic techniques to simplify the scheduling process. The system is designed using a combination of the **Greedy Algorithm** and constraint-handling methods. The greedy approach helps in selecting the most suitable subject for each time slot based on current conditions, while constraint tracking ensures that the generated timetable follows all necessary rules.

The system accepts inputs such as subject details, faculty names, classroom information, and required weekly hours. Based on these inputs, it automatically generates a timetable for multiple days and periods. The generated timetable is structured, easy to understand, and free from common scheduling conflicts.

Overall, this project aims to reduce manual effort, improve efficiency, and provide a reliable solution for timetable generation. It demonstrates how basic algorithmic concepts can be applied to solve practical problems in academic environments.

Main Aim of the Project

The main aim of this project is to develop a **Smart Class Timetable Generator** that can automatically create an efficient and conflict-free timetable using algorithmic approaches. The system focuses on reducing manual effort, ensuring proper allocation of subjects, faculty, and classrooms, and maintaining a balanced schedule by applying **Greedy Algorithm and constraint-handling techniques**.

Objectives of the Project

- To design and develop an automated system for generating class timetables.
- To minimize manual work and reduce human errors in timetable preparation.
- To ensure fair and balanced distribution of subjects across all working days.
- To avoid consecutive repetition of the same subject in a single day.
- To allocate faculty and classrooms efficiently without conflicts.
- To satisfy constraints such as required number of periods per subject per week.
- To improve overall scheduling efficiency using greedy-based decision making.
- To make the timetable clear, structured, and easy to understand.
- To demonstrate the practical application of algorithms in solving real-world problems.

Methodology

The Smart Class Timetable Generator is developed using a structured approach that combines the **Greedy Algorithm** with **constraint-handling techniques inspired by Dynamic Programming**. The methodology focuses on generating an efficient and conflict-free timetable by following a step-by-step process.

1. Input Collection

The system begins by collecting necessary inputs from the user, including:

- Subject names
- Faculty assigned to each subject
- Classroom details
- Required number of periods per week for each subject

These inputs form the base data required for timetable generation.

2. Initialization

A timetable structure is created using a two-dimensional format representing **days and periods**. Initially, all slots are marked as "Free." A separate tracking structure is also maintained to monitor how many times each subject is scheduled per day.

3. Greedy Allocation Approach

The core of the system uses a greedy strategy. For each time slot:

- Subjects are randomly shuffled to ensure fairness
- The algorithm selects the best possible subject that satisfies all constraints
- The selected subject is assigned immediately to the current slot

This ensures quick decision-making while maintaining efficiency.

4. Constraint Handling

While assigning subjects, the system checks multiple constraints:

- A subject should not exceed its required weekly periods
- A subject should not be assigned more than a limited number of times per day
- Consecutive repetition of the same subject is avoided
- A fixed slot is reserved for breaks (e.g., lunch break)

These constraints ensure that the timetable remains practical and balanced.

5. Dynamic Tracking (DP Concept)

The system maintains and updates tracking arrays (such as subject hours and daily counts). This helps in:

- Remembering past allocations
- Preventing conflicts
- Making better decisions for future slots

Although not full dynamic programming, this approach uses **state tracking**, which is a core idea of DP.

6. Timetable Generation

The process continues until all days and periods are filled. If no subject satisfies the constraints for a slot, it is marked as “Free.”

7. Output Display

Finally, the generated timetable is displayed in a clear and readable format, showing:

- Day-wise schedule
- Period-wise subject allocation
- Faculty and room details.

6.S/W AND H/W REQUIREMENTS

To implement and execute the Daily Habit Tracker, the following resources are required:

Software Requirements:

Component	Specification
Programming Language	Python (Version 3.x or above)
Compiler	Visual Studio Code / IDLE / PyCharm
Operating System	Windows / Linux / macOS
Required Libraries	iostream, vector, string, algorithm, random

Hardware Requirements:

Component	Specification
Processor	Intel i3 or above
RAM	Minimum 4 GB
Storage	At least 100 MB free space
Input Device	Keyboard and Mouse
Output Device	Monitor

CODE

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <random>
using namespace std;
int main() {
    int days = 5;
    int periods = 6;
    int subjects;
    cout << "===== Smart Class Timetable Generator =====" << endl;
    cout << "Enter number of subjects: ";
    cin >> subjects;
    cin.ignore();
    vector<string> subjectNames(subjects);
    vector<string> facultyNames(subjects);
    vector<string> roomNames(subjects);
```

```
vector<int> subjectHours(subjects);
for (int i = 0; i < subjects; i++) {
    cout << "\nEnter subject name " << i + 1 << ": ";
    getline(cin, subjectNames[i]);
    cout << "Enter faculty name for " << subjectNames[i] << ": ";
    getline(cin, facultyNames[i]);
    cout << "Enter room name for " << subjectNames[i] << ": ";
    getline(cin, roomNames[i]);
    cout << "Enter required periods per week for " << subjectNames[i] << ": ";
    cin >> subjectHours[i];
    cin.ignore();
}
vector<vector<string>> timetable(days, vector<string>(periods, "Free"));
vector<vector<int>> dailyCount(days, vector<int>(subjects, 0));
string dayNames[5] = {
    "Monday", "Tuesday", "Wednesday", "Thursday", "Friday"
};
random_device rd;
mt19937 g(rd());
for (int i = 0; i < days; i++) {
    for (int j = 0; j < periods; j++) {
        if (j == 3) {
            timetable[i][j] = "Lunch Break";
            continue;
        }
        vector<int> order(subjects);
        for (int k = 0; k < subjects; k++) order[k] = k;
        shuffle(order.begin(), order.end(), g);
        bool assigned = false;
        for (int idx : order) {
            if (subjectHours[idx] <= 0) continue;
            if (dailyCount[i][idx] >= 2) continue;
            if (j > 0 && timetable[i][j - 1].find(subjectNames[idx]) != string::npos)
                continue;
            timetable[i][j] =
                subjectNames[idx] + " | " +
                facultyNames[idx] + " | " +
```

```
                roomNames[idx];
            subjectHours[idx]--;
            dailyCount[i][idx]++;
            assigned = true;
            break;
        }
        if (!assigned) {
            timetable[i][j] = "Free";
        }
    }
}
cout << "\n===== Generated Smart Class Timetable =====\n" << endl;
for (int i = 0; i < days; i++) {
    cout << dayNames[i] << ":" << endl;
    for (int j = 0; j < periods; j++) {
        cout << "Period " << j + 1 << " -> " << timetable[i][j] << endl;
    }
    cout << endl;
}
cout << "Timetable generated using Greedy + Constraints!" << endl;
return 0;
}
```

```
===== Smart Class Timetable Generator =====
Enter number of subjects: 3

Enter subject name 1: coding skills 2
Enter faculty name for coding skills 2: prakash sir
Enter room name for coding skills 2: C202
Enter required periods per week for coding skills 2: 3

Enter subject name 2: Maths
Enter faculty name for Maths: Tapan kumar sir
Enter room name for Maths: S404
Enter required periods per week for Maths: 3

Enter subject name 3: DBMS
Enter faculty name for DBMS: Abdulsami
Enter room name for DBMS: S407
Enter required periods per week for DBMS: 5

===== Generated Smart Class Timetable =====

Monday:
Period 1 -> Maths | Tapan kumar sir | S404
Period 2 -> coding skills 2 | prakash sir | C202
Period 3 -> Maths | Tapan kumar sir | S404
Period 4 -> Lunch Break
Period 5 -> DBMS | Abdulsami | S407
```

```
Period 6 -> coding skills 2 | prakash sir | C202

Tuesday:
Period 1 -> DBMS | Abdulsami | S407
Period 2 -> coding skills 2 | prakash sir | C202
Period 3 -> DBMS | Abdulsami | S407
Period 4 -> Lunch Break
Period 5 -> Maths | Tapan kumar sir | S404
Period 6 -> Free

Wednesday:
Period 1 -> DBMS | Abdulsami | S407
Period 2 -> Free
Period 3 -> DBMS | Abdulsami | S407
Period 4 -> Lunch Break
Period 5 -> Free
Period 6 -> Free

Thursday:
Period 1 -> Free
Period 2 -> Free
Period 3 -> Free
Period 4 -> Lunch Break
Period 5 -> Free
Period 6 -> Free
```

```
Period 2 -> Free
Period 3 -> DBMS | Abdulsami | S407
Period 4 -> Lunch Break
Period 5 -> Free
Period 6 -> Free

Thursday:
Period 1 -> Free
Period 2 -> Free
Period 3 -> Free
Period 4 -> Lunch Break
Period 5 -> Free
Period 6 -> Free

Friday:
Period 1 -> Free
Period 2 -> Free
Period 3 -> Free
Period 4 -> Lunch Break
Period 5 -> Free
Period 6 -> Free

Timetable generated using Greedy + Constraints!


=== Code Execution Successful ===
```

Conclusion

The Smart Class Timetable Generator successfully demonstrates how algorithmic techniques can be used to solve real-world scheduling problems in an efficient manner. By using a combination of the **Greedy Algorithm** and constraint-handling methods, the system is able to generate a structured and conflict-free timetable with minimal manual effort.

The project ensures proper distribution of subjects, avoids unnecessary repetition, and effectively allocates faculty and classroom resources. It also reduces the complexity and time involved in manual timetable preparation, making the process faster and more reliable.

Although the current system works well for basic scheduling, it can be further enhanced by incorporating additional features such as faculty availability constraints, user-friendly graphical interfaces, and advanced optimization techniques.

Overall, this project highlights the practical application of programming and problem-solving skills in developing efficient solutions for academic management. It proves that even simple algorithms, when applied correctly, can significantly improve productivity and accuracy in everyday tasks.